OV VO

# Getting OVVO onto a real account.

Everything I need from you, in the order you need it, with the exact place each value lives and the exact way to give it to me. Nothing here costs money and nothing here needs a card.

**9**
PARTS

**30 min**
OF YOUR TIME

**2**
FREE ACCOUNTS

| | |
|---|---|
| 1 | Make a Firebase project, so a traveller can sign in with an email or a Google account. |
| 2 | Register the web app and copy six public values, which is the whole of what I need from Firebase. |
| 3 | Turn on email and password sign-in. |
| 4 | Turn on Google sign-in and publish the consent screen, so strangers are allowed in. |
| 5 | Tell Firebase that the workers.dev address is allowed to start a sign-in. |
| 6 | Make a Cloudflare account and one scoped token, so I can put the site and the database up. |
| 7 | Send me the values in one paste, and put the one secret where secrets go. |
| 8 | Check it works with your own two sign-ins before I build on it. |
| 9 | What happens after that, and what I promise about your year. |

OVVO · setup guide · written for the steerer, not for the code

PART 1

# Make the Firebase project

Firebase is Google's sign-in service. The free plan is permanent, needs no card, and covers far more travellers than this site will ever have. One project is enough for development, the preview and the live site.

## 1.1 Sign in to the Firebase console

WHERE — https://console.firebase.google.com — sign in with the Google account you want to own OVVO. Do not use a work or school account: those can be closed by an administrator and take the project with them.

## 1.2 Create the project

Click **Create a project** (the button on the welcome card, or **Add project** at the top).

Type a name. **OVVO** is fine; the name is only what you see in the console.

COPY THIS — Click the pencil icon beside the auto-generated **project ID** and change it to something clean, for example `ovvo-app`. This ID appears in a public web address later and **cannot be changed afterwards**.

## 1.3 Turn Analytics off

The next screen offers Google Analytics. It is marked optional and we do not use it, so switch the toggle **off**.

Accept the terms if it asks, then click **Create project** and wait for the spinner to finish. Click **Continue** when it offers.

CAREFUL — Do not click anything that offers to "upgrade" the project. Firebase's paid platform changes the free limits from generous to three thousand people a day, and there is no way back that is worth the trouble.

PART 2

# Register the web app and copy six values

This is the only thing I need out of Firebase. All six values are public by design: they identify the project, they do not unlock it. Google writes this itself, and the site ships them in the page exactly as every Firebase app does.

## 2.1 Add a web app

On the project overview page, click the **web** icon (it looks like a small screen with a bracket, labelled </>).

Give it a nickname, for example **OVVO web**. Leave the Firebase Hosting checkbox **unticked**: we are not using their hosting.

Click **Register app**.

## 2.2 Copy the config block

The next screen shows a block that begins `const firebaseConfig = {`. Copy the whole block into a note for later, or leave the tab open.

COPY THIS — The six values I need: `apiKey`, `authDomain`, `projectId`, `storageBucket`, `messagingSenderId` and `appId`.

## 2.3 If you lose it, it is always here

WHERE — Gear icon → **Project settings** → **General** tab → scroll to **Your apps** → your web app → **SDK setup and configuration** → **Config**. The same block appears, at any time, forever.

## 2.4 Two of the values do double duty

projectId is also on that General tab, in the top pane, labelled **Project ID**.

authDomain is normally your-project-id.firebaseapp.com. Keep it exactly as Firebase gave it to you. If you change it to your own address, Google sign-in stops working until extra Google Cloud setup is done, and we do not need that trade.

PART 3

# Turn on email and password

This is the sign-in that works with nothing but an address and a password, and it is the one that has to work on a new phone after a forgotten password.

## 3.1 Open Authentication

WHERE — Left sidebar → **Security** → **Authentication**. (Some consoles show **Build** → **Authentication**. Either way it is the item called Authentication.)

## 3.2 Enable the provider

Open the **Sign-in method** tab. In the list of providers click **Email/Password**. Turn the first toggle on. Leave **Email link (passwordless sign-in)** off for now.

Click **Save**.

## 3.3 Do not add anything else

Ignore phone, anonymous and the rest. Every provider you switch on is another way in and another thing to keep right.

Firebase sends its own password reset and verification emails on the free plan, so you do not need an email service, a domain, or any setup for that.

PART 4

# Turn on Google sign-in

One tap with the Google account already on the phone. This is the part with the most hidden steps, so it has the most detail.

## 4.1 Enable the provider

Still in **Authentication → Sign-in method**, click **Google** and turn its toggle on.

It asks for a **project public-facing name** (type **OVVO**) and a **support email** (pick yours from the dropdown).

Click **Save**.

COPY THIS — You do **not** need to create a Google OAuth client. Firebase keeps its own for you, which is why we leave `authDomain` alone in part 2.

## 4.2 Set up the consent screen once

WHERE — https://console.cloud.google.com — the same project is there under the name you gave it. Open the menu and choose **Google Auth platform**. If it says it is not configured, click **Get Started**.

## 4.3 Fill in the four things it asks for

**App name:** OVVO. **User support email:** your address. Click **Next**.

**Audience:** choose **External**. This sounds public, and that is correct: it means anyone with a Google account, not only people inside an organisation. Click **Next**.

**Contact information:** your address again. Click **Next**.

Tick the box agreeing to the user data policy, click **Continue**, then **Create**.

## 4.4 Publish it

On the **Audience** page you will see **Publishing status: Testing**. Click **Publish app** and confirm.

This matters more than it looks. While it says Testing, only a hundred named test accounts can sign in, and their sign-ins expire after seven days.

CAREFUL — Publishing with only the basic profile and email scopes does **not** need Google's review. The app still works immediately; the only thing that may lag is OVVO's name and logo showing on the consent screen, which is cosmetic.

PART 5

# Allow the workers.dev address

Firebase refuses to start a sign-in from an address it does not know, even a good one. This is the step that decides whether the whole plan works, so it is also the first thing I will test.

## 5.1 Find the list

WHERE — **Authentication → Settings** tab → scroll to **Authorized domains**. This is the Settings tab, not the Sign-in method tab; older guides get this wrong.

## 5.2 Add the site address

Click **Add domain** and type the bare host name, with no https:// and no slash:

COPY THIS — For example `ovvo.something.workers.dev`. If you do not know the exact address yet, skip this and come back once I have deployed the first version; I will tell you the exact string to paste.

## 5.3 Add localhost

Click **Add domain** again and add localhost, on its own.

Projects made after April 2025 no longer include it, and without it nothing works on your own machine while I am building.

## 5.4 Leave the two Firebase addresses alone

You will see your-project-id.firebaseapp.com and your-project-id.web.app already in the list. Do not remove them: they are where the sign-in window actually travels through, even when the popup is opened from our own address.

PART 6

# Make the Cloudflare account and one token

Cloudflare will hold the site, the small API and the database, all on one free plan with no card and nothing that ever sleeps. To put it up from here I need a token, and this is the only secret in the whole guide.

## 6.1 Make the account

WHERE — https://dash.cloudflare.com/sign-up — an email and a password is all it asks for. No card, and nothing to pay for what we are using.

## 6.2 Copy the account ID

WHERE — **Workers & Pages** on the left. On that page, in the **Account details** panel on the right, there is **Account ID** with a copy icon. It is a long string of letters and numbers.

COPY THIS — The account ID is **not** a secret. Send it to me in the paste in part 7.

## 6.3 Create the token

Click your profile icon, top right, then **Profile**, then **API tokens** in the left menu. (On the newer dashboard this is **Manage Account → Account API tokens**.)

Click **Create Token**, then **Create Custom Token** at the bottom of the template list, and name it **OVVO deploy**.

## 6.4 Give it exactly three permissions

Under **Permissions**, add these three rows, one at a time, each with **Account** as the scope and your account chosen under **Include**:

**Workers Scripts → Edit**, then **D1 → Edit**, then **Account Settings → Read**.

CAREFUL — The pre-made "Edit Cloudflare Workers" template does **not** include D1. If you pick it, the deploy fails later with a permission error, so use the custom one.

## 6.5 Copy it once, and put it where secrets go

Click **Continue to summary**, then **Create Token**. The value is shown **once**. It starts with cfut_.

Paste it into this project under **Settings → Secrets**, as a secret named CLOUDFLARE_API_TOKEN. Add CLOUDFLARE_ACCOUNT_ID there too, with the account ID from step 6.2.

CAREFUL — Never paste the token into a chat message, a document, a screenshot or a code file. If it ever lands somewhere it should not, come back to the token list, delete it, and make a new one. Nothing is lost by doing that.

PART 7

# What to send me

One message, one paste block. Everything below is safe to send; nothing below unlocks anything on its own.

### Copy this, fill it in, send it

Leave a line blank if you have not done that part yet. I will tell you what is missing rather than guessing.

```
Firebase project ID:

Firebase apiKey:

Firebase authDomain:

Firebase storageBucket:

Firebase messagingSenderId:

Firebase appId:

Email + password turned on?      yes / no

Google turned on?                yes / no

Consent screen published?        yes / no

workers.dev address, if known:

Cloudflare account ID:

Secrets added: token + account ID  yes / no
```

CAREFUL The Cloudflare token does not go in this block. It lives only in Settings → Secrets, and I can see that it is there without ever seeing its value.

PART 8

## Check it before I build on it

Five minutes now saves a confusing afternoon later. Do these once the paste is sent; I will be doing the same checks from my side.

- [ ] In **Authentication → Sign-in method**, both Email/Password and Google show as Enabled.
- [ ] In **Authentication → Settings → Authorized domains**, your workers.dev address and localhost are both listed.
- [ ] In the Google console, **Google Auth platform → Audience** says **In production**, not Testing.
- [ ] In Firebase, **Project settings → General** still shows your web app, and the config block opens.
- [ ] In Cloudflare, the token exists in the list and is called OVVO deploy.
- [ ] Under this project's Settings → Secrets, both names appear with a value set.

Then, once I have the first version up, the real test is four actions on your own phone: sign in with an email and password, sign out, sign in with Google, and confirm the year on screen is still the same year.

PART 9

## If something goes wrong

These are the three failures this setup actually produces, what they look like, and what fixes them. None of them is a broken build.

> **`auth/unauthorized-domain`**
> The sign-in refuses to start and names the address. Part 5 was skipped, or the address was typed with https:// in front of it. Add the bare host name.

> **`Error 400: origin_mismatch`**
> The Google window opens and then blocks. This one belongs to a Google OAuth client, which we deliberately do not use. It appears only if `authDomain` was changed in the Firebase config. Put back the `firebaseapp.com` value from part 2.

> **`The popup opens and closes, or the browser blocks it`**
> Google sign-in from a site that is not hosted by Firebase has to use the popup, and some browsers are strict about it. A second click normally works; if it never does, I add a small proxy in the Worker so the redirect route works too.

> **`A reset email never arrives`**
> On the free plan the mail comes from a `firebaseapp.com` address, and spam filters are wary of it. Check the spam folder before assuming the setup is wrong. If it becomes a habit, the fix is a mail domain, and that is the one thing in this plan that costs money.

PART 10

# After that

So you know what you are waiting for, and what I will not do with any of it.

**What I build next.** The site and a small API on Cloudflare, a database table for the year, the dated tally underneath the Wrapped, and an account card in Settings, not in the top row. The whole app keeps working signed out.

**What the database holds.** Dated events: a journey planned, a board opened, a stop hearted, a theme worn. They belong to an account and nobody else can read them, because every query is filtered by the signed-in account on the server.

**What it never holds.** Your name, your email, your location, your password. Firebase keeps the account; the database keeps the year and only the identifier Firebase gives it.

**The controls that go with it.** One to forget a year, one to delete everything the account holds, and a plain sentence in Settings saying what leaves the device.

**The honest limit.** A year can only be counted from the day the tally starts. The first December will be a short year, and the card will say so rather than dressing up three journeys as a life.